

BEANS — Master Roadmap & Solution Architecture

Bitcoin Encryption, Analysis & Network Security

SIH PS 26146 · AI-Powered Monitoring & Analysis of Bitcoin Transaction Traffic · NTRO

What this file is. It is the single source of truth for the team. It covers what the problem statement actually asks for, what our existing docs get right and wrong, the final architecture, every feature (prioritised), the ML design, the synthetic-data plan, the repo layout, the week-by-week build plan, the demo script, and the write-up outline.

Repo: [git@github.com:KAVYAJOSHI1/BEANS.git](https://github.com/KAVYAJOSHI1/BEANS.git) · Source docs: [docs/archive/](#) (approach.md, setup.md, SIH_PS_26146.pdf)

0. TL;DR

We build **BEANS (Bitcoin Encryption, Analysis & Network Security)**, an **offline, Linux-native** forensic pipeline:

```
CSV/JSON/XML → Ingest + Validate + GeoIP/ASN enrich → Heterogeneous graph (IP ↔ TX ↔ Wallet ↔ ASN)
  → 4 ML engines (Clustering · Anomaly · Peel/Mix · Risk propagation)
  → Calibrated fusion score + SHAP explanations
  → Ranked, explainable alert list → Dashboard (alerts · link graph · timeline · evidence) + PDF case report
```

Five decisions that shape everything else:

1. **ML decides and rules only produce features.** The PS says a "*working model — not just rules.*" Every heuristic (CoinJoin shape, peel pattern, geo-jump) becomes an input feature to a trained model. No heuristic is used as a verdict on its own.
 2. **We generate the dataset ourselves.** The PS PDF says *Dataset Link: Nil*. A parameterised synthetic generator with **ground-truth labels** is therefore a core deliverable, and it is also what lets us report precision and recall numbers.
 3. **Everything runs fully offline.** There are no live OSINT feeds and no downloads at runtime. The GeoIP DB, model weights, Python wheels, and frontend build all ship inside the repo or the release bundle.
 4. **The stack stays light enough to demo on a laptop.** We use DuckDB + NetworkX/igraph + PyTorch Geometric, served by FastAPI, with a React + Cytoscape.js UI. Neo4j and Postgres are dropped (reasons in §2).
 5. **Every flag carries an explanation.** Each alert has a score, a calibrated confidence, its top SHAP reasons in plain English, and the evidence subgraph.
-

1. The Problem Statement, decoded

1.1 Mandatory deliverables (from the PDF)

#	PS requirement	Our component	Proof we show judges
R1	Ingest & parse bulk metadata in CSV/JSON/XML	<code>beans.ingest</code> (3 parsers → one canonical schema)	Load the same dataset in all 3 formats; identical row counts
R2	Fields: timestamp, src/dst IP & port, TXID, in/out addresses, amounts, fee, script type	Canonical schema §5.1	Schema validation report
R3	geo_country / ASN from an open-source downloadable GeoIP DB	DB-IP Lite (country + ASN, <code>.mmdb</code> , CC-BY 4.0) bundled offline	Enrichment coverage %
R4	Entity/transaction graph linking IPs, wallets, transactions	<code>beans.graph</code> heterogeneous graph	Interactive link-analysis view
R5	AI/ML detection with a working model, not just rules	4 engines (§6) + fusion model	Metrics on held-out synthetic data
R6	Ranked, explainable alert list with confidence score	Fusion + SHAP + calibration	Alert table with "Why flagged"
R7	Dashboard / link-analysis visualisation with evidence per flag	React + Cytoscape.js UI	Live demo
R8	Workable complete offline solution for Linux	Offline bundle + <code>make demo</code>	Demo with Wi-Fi turned OFF
R9	Working prototype (code repo) with ingestion, correlation, AI/ML	This repo	—
R10	Short technical write-up: approach, model choice, explainability	<code>docs/TECHNICAL_WRITEUP.md</code> (§12)	PDF handout

1.2 The official AI/ML focus-area table (from page 9 of the PDF; missing from the portal text)

Focus area	What NTRO wants built	Our engine
Entity Clustering	Group wallets likely owned by one entity using common-input-ownership + graph embeddings	E1: CIOH union-find + change heuristic → GraphSAGE/node2vec embeddings → HDBSCAN
Anomaly Detection	Flag statistically unusual transactions/flows	E2: Isolation Forest + (optional) autoencoder on tx and wallet features
Peeling-Chain / Mixing Detection	Detect laundering sequences (peeling chains, CoinJoin-like structures)	E3: sequence/structure features → LightGBM typology classifier
Risk Scoring	Propagate risk scores from seed illicit wallets via algorithms	E4: personalised PageRank / decayed taint + GNN node classifier

All four need to be visibly present in the demo. The judges will have this exact table in front of them.

1.3 Implicit requirements (read between the lines)

- **Network ↔ blockchain correlation is the differentiator.** Most teams will do chain analysis only. Linking the IP that *first* relayed a TX to that TX's wallets (the "first-spy" estimator) is the NTRO-flavoured part, so we treat it as a first-class feature.

- **Scale:** "bulk" means we must handle $\geq 1M$ rows on a laptop. Benchmark it.
- **Investigator use:** results need to be defensible, so we keep an audit log and exportable evidence.

2. Review of our existing docs (`approach (1).md`, `setup.md`)

Keep

- Three data layers (blockchain / network / correlation metadata).
- **Pattern library** (ransomware, CoinJoin, exchange consolidation, darknet, hack laundering). We reuse it as the **typology spec for the synthetic generator and the label set**.
- Alert / Case / Audit-log data model, the evidence-report template, and the dashboard views (alert ranking, entity card, case file, timeline).
- Success metrics section.

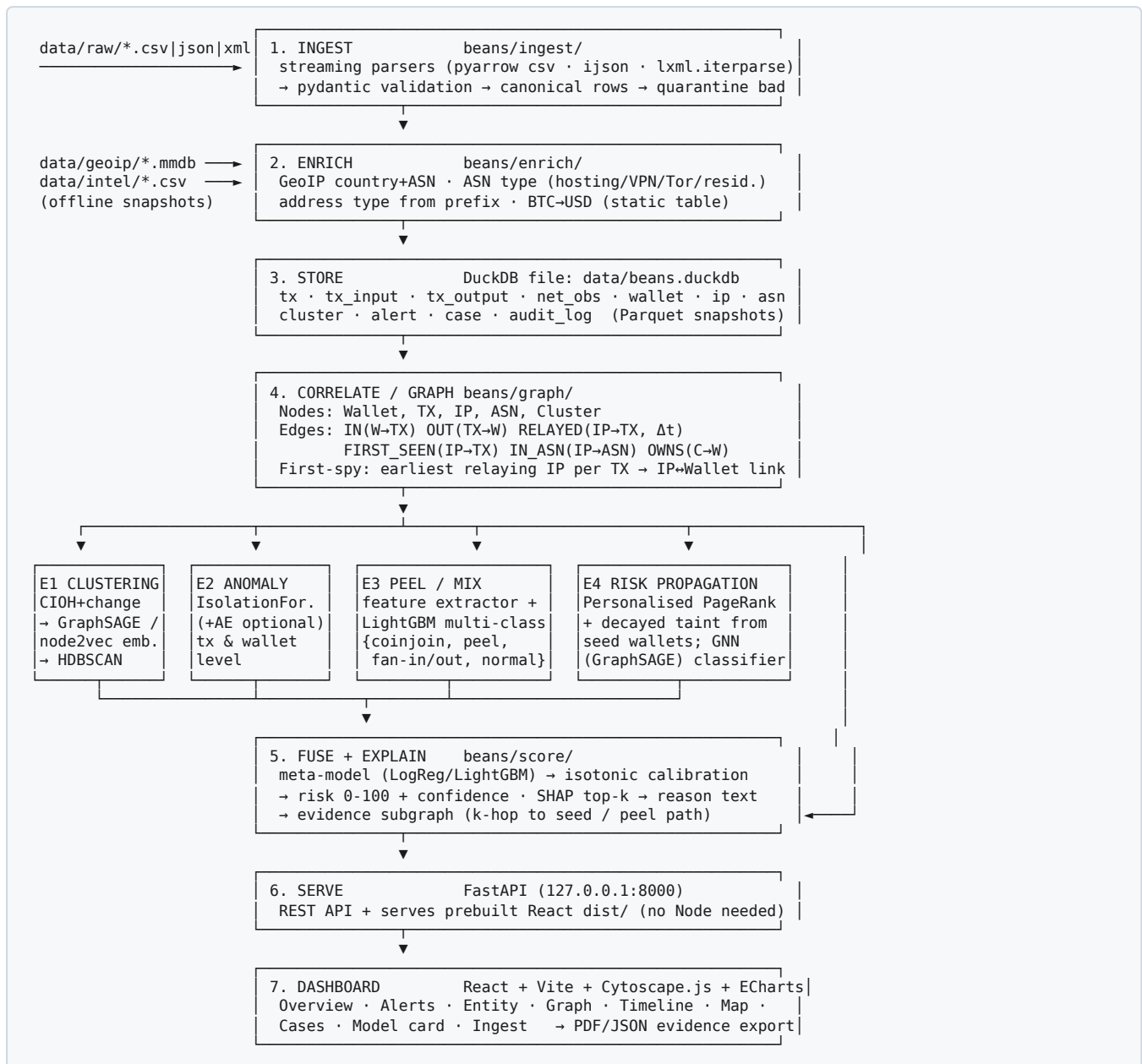
Fix (these would cost us marks or break the demo)

Problem	Where	Fix
Rule-based, "not algorithmic automation" philosophy	approach Part 1/3	Directly contradicts R5. Rules become features and ML models decide (§6).
Relies on online OSINT (Chainalysis, Elliptic, threat feeds)	approach 3.1-B, 4.1	Violates R8. Use a local seed-list file (<code>data/seeds/illicit_seeds.csv</code>) plus offline Tor-exit/hosting-ASN lists snapshotted into the repo.
Setup needs internet (<code>apt</code> , <code>docker pull neo4j:latest</code> , <code>npm</code> , <code>pip</code>)	setup Phase 1-4	Offline bundle: <code>pip wheelhouse</code> , prebuilt UI <code>dist/</code> , <code>docker save tarball</code> (§10).
Invalid PostgreSQL DDL: inline <code>INDEX idx (...)</code> inside <code>CREATE TABLE</code> is MySQL syntax	setup 2.2	Moot, since we drop Postgres. If kept, use separate <code>CREATE INDEX</code> .
No XML ingestion	setup 3.6	Add <code>lxml</code> streaming parser.
GeoIP only named (<code>geoip2</code>), never integrated; <code>MaxMind-DB-Writer</code> unnecessary	setup 3.2	DB-IP Lite <code>.mmdb</code> + <code>maxminddb</code> reader, offline.
No synthetic data generator, but the PS dataset is Nil	—	New <code>beans.synth</code> module (§5).
No explainability method, no confidence calibration	—	SHAP + isotonic calibration (§7).
No peel-chain detection, no risk propagation from seeds	—	E3, E4.
Neo4j + Postgres + Flask + CRA: 4 servers, JVM, heavy install	setup	DuckDB (embedded) + in-process graph + FastAPI + Vite. One process, one command.
<code>cytoscape-react</code> isn't the real package; CRA is deprecated	setup 4.2	<code>react-cytoscapejs</code> + <code>cytoscape</code> , scaffold with Vite.
<code>NEO4JLABS_PLUGINS</code> env var (old), hard-coded passwords, <code>CORS(*)</code>	setup	Moot, or fixed with a <code>.env</code> and a localhost-only bind.

Neo4j as an option: we keep a `beans export --neo4j` command that writes CSVs for `neo4j-admin import`, so any judge who asks "can it scale to a graph DB?" gets a yes without us running Neo4j

during the demo.

3. Final Architecture



3.1 Tech stack (final)

Layer	Choice	Why
Language	Python 3.11	ML ecosystem
Parsing	pyarrow/polars (CSV), ijson (JSON stream), lxml.iterparse (XML stream)	Streaming, so 1M+ rows fit in RAM
Validation	pydantic v2	Typed schema and clear error rows
Storage	DuckDB (+ Parquet)	Embedded, zero-admin, fast analytics, one file, offline
Graph	networkx for logic, igraph for speed on big graphs, PyTorch Geometric for GNN	In-process, no DB server

Layer	Choice	Why
ML	scikit-learn, lightgbm, hdbscan, torch (CPU) + torch_geometric	CPU-only is fine
Explainability	shap (TreeExplainer), PyG GNNExplainer, path-based evidence	Required by R6/R10
GeoIP	DB-IP Lite country + ASN .mmdb via maxminddb	Free, no account, CC-BY. (MaxMind GeoLite2 needs a license key, so keep it only as an alternative.)
API	FastAPI + uvicorn	Async, auto OpenAPI docs
UI	React 18 + Vite + TypeScript, cytoscape + react-cytoscapejs, ECharts, Tailwind	Built once to static dist/ and served by FastAPI
Reports	Jinja2 → HTML → PDF (weasyprint)	Offline evidence pack
CLI	typer	beans ingest / train / score / serve / demo
Packaging	Makefile + pip wheelhouse + optional Docker image tar	Offline install
Quality	pytest, ruff, GitHub Actions	—

4. Feature List (MoSCoW)

MUST (MVP, needed to qualify)

- ❑ **M1** Streaming ingest for CSV, JSON (array + NDJSON), and XML → canonical schema; bad rows go to quarantine.csv with a reason
- ❑ **M2** Array-field handling (input_addresses[] etc.) in all 3 formats; `sum(in) - sum(out) = fee` consistency check
- ❑ **M3** Offline GeoIP country + ASN enrichment for src/dst IP
- ❑ **M4** Heterogeneous graph IP ↔ TX ↔ Wallet ↔ ASN, plus first-spy IP→TX attribution
- ❑ **M5** E1 Entity clustering (CIOH + embeddings)
- ❑ **M6** E2 Anomaly detection (Isolation Forest)
- ❑ **M7** E3 Peel-chain + CoinJoin detection (ML classifier)
- ❑ **M8** E4 Risk propagation from seed wallets
- ❑ **M9** Fusion → **risk 0-100 + calibrated confidence**, ranked alert list
- ❑ **M10** SHAP-based **"Why flagged"** reasons (plain English) per alert
- ❑ **M11** Dashboard: alert table, entity page with evidence, link-analysis graph
- ❑ **M12** Synthetic dataset generator with ground-truth labels, in all 3 formats
- ❑ **M13** Runs with networking disabled; one-command demo `make demo`
- ❑ **M14** Technical write-up + README

SHOULD (the "winning" layer)

- ❑ **S1** Timeline view: fund flow over time, with peel chain animated hop by hop
- ❑ **S2** Geo map: IP origins, "impossible travel" arcs (offline world GeoJSON)
- ❑ **S3** Evidence pack export (PDF + JSON with SHA-256 hash of the input file, for chain of custody)
- ❑ **S4** Case management: group alerts into a case, add notes, mark status, audit log
- ❑ **S5** Investigator feedback loop: mark a false positive → gets stored → used in the next `beans train` run (active learning)
- ❑ **S6** Model card page: metrics per typology, PR curves, confusion matrix, feature importance

- ☐ **S7** Graph filters: min risk, typology, country, ASN type, time window; expand k-hop on click
- ☐ **S8** Seed management UI: upload a seed list, re-propagate risk live
- ☐ **S9** Performance benchmark: 1M rows ingest + score time on a laptop

COULD (only if time remains)

- ☐ **C1** GNN (GraphSAGE) node classifier with GNNExplainer (upgrades E4)
- ☐ **C2** Temporal/sequence model for flows (e.g. GRU over a wallet's tx sequence)
- ☐ **C3** Local LLM (e.g. llama.cpp + small quantised model) that turns the evidence JSON into a narrative report. It must stay offline; skip it if it's risky.
- ☐ **C4** Watch-folder mode: drop a file in `data/inbox/` and it gets auto-ingested and scored (the "monitoring" in the PS title)
- ☐ **C5** Neo4j export, STIX 2.1 export of indicators
- ☐ **C6** RBAC/login (two roles: analyst, supervisor)
- ☐ **C7** Validate E3/E4 on the public **Elliptic** dataset (downloaded beforehand) as an external benchmark

WON'T (explicitly out of scope; say so in the write-up)

- Live P2P sniffing / live node connection, and online threat feeds (they violate the offline requirement)
- Altcoin / cross-chain tracing
- Real de-anonymisation of real people (synthetic data only)

5. Data: Canonical Schema & Synthetic Generator

5.1 Canonical input record (one row = one network observation of a TX)

Field	Type	Req	Notes
timestamp	ISO-8601 UTC	✓	observation time
src_ip, dst_ip	IPv4/IPv6 str	✓	
src_port, dst_port	int	✓	8333 = BTC P2P; 18333 testnet; others are anomalous
txid	64-hex	✓	
input_addresses[], output_addresses[]	list[str]	✓	CSV: ; -joined or a JSON-string cell. Support both.
input_amounts[], output_amounts[]	list[float BTC]	✓	same length as the address lists
fee	float BTC	✓	
script_type	enum P2PKH/P2SH/P2WPKH/P2WSH/P2TR	✓	
geo_country, asn	str	enrich	filled by us if missing
block_height, bytes, protocol	—	opt	

The same TXID can appear in many rows (seen from many peers). The earliest `src_ip` per TXID is the **first-spy origin candidate**.

Normalised tables (DuckDB): tx(txid, ts_first_seen, fee, script_type, n_in, n_out, total_in, total_out), tx_input(txid, idx, address, amount), tx_output(txid, idx, address, amount), net_obs(id, ts, src_ip, src_port, dst_ip, dst_port, txid, country, asn, asn_type), wallet(address, first_seen, last_seen, n_tx, recv, sent, cluster_id, ...), ip(ip, country, asn, asn_type, n_tx_first_seen), alert, case, audit_log, feedback.

5.2 Formats

CSV

```
timestamp,src_ip,src_port,dst_ip,dst_port,txid,input_addresses,input_amounts,output_addresses,output_amounts,fee,script_type
2026-03-
01T10:02:11Z,185.220.101.4,51234,94.23.1.7,8333,9f2c...e1,"bc1qa;bc1qb","0.5;0.7","bc1qx;bc1qy","1.19;0.0095",0.0005,P2WPKH
```

JSON: an array or NDJSON of the same objects, with real arrays. **XML**

```
<transactions><tx txid="9f2c...e1" timestamp="..." fee="0.0005" script_type="P2WPKH">
  <net src_ip="185.220.101.4" src_port="51234" dst_ip="94.23.1.7" dst_port="8333"/>
  <inputs><in address="bc1qa" amount="0.5"/>...</inputs>
  <outputs><out address="bc1qx" amount="1.19"/>...</outputs></tx></transactions>
```

Document these in docs/DATA_FORMATS.md, and make parsers tolerant of the judges' own column names via a --mapping mapping.yaml option. **This matters a lot: at the finale NTRO may hand you their own file.**

5.3 Synthetic generator beans synth (core deliverable, since the dataset is Nil)

Principles: agent-based simulation with a real UTXO ledger (outputs get spent later, so the graph is realistic), realistic address formats per script type, fees from a fee-rate distribution × tx vsize, and diurnal timing per actor time zone.

Actors (legit, ~95%) | Actor | Behaviour | |---|---| | Retail users | 1-3 inputs, 2 outputs (payment + change), residential ASN, stable country | | Exchanges | deposit address per user → periodic **consolidation** sweeps; hot-wallet **batch payouts** (1 in, 50-200 out); datacenter ASN | | Merchants / payment processors | many small inbound, periodic sweep | | Miners/pools | coinbase-like fan-out payouts |

Illicit typologies (~3-5%, labelled), taken from the approach doc's pattern library: | Label | Chain signature | Network signature | |---|---|---| | RANSOMWARE | fresh wallet receives ransom-sized amount → holds 24-48h → splits 5-20 → mix → exchange | Tor exit / VPN / bulletproof-hosting ASN, off-hours broadcast | | PEEL_CHAIN | 1 in → 2 out (small "peel" + large remainder), remainder re-spent within minutes, chain length 5-50 | same IP/ASN relays consecutive hops | | COINJOIN | n_in ≥ 5 unrelated, ≥ 5 **equal-value** outputs (0.01/0.05/0.1 BTC denominations) + change | coordinator-like IP pattern | | DARKNET_MARKET | fan-in from 100+ buyers → daily seller withdrawals, P2SH/P2WSH 2-of-3 multisig | hosting ASN, regular 24h cycle | | HACK_LAUNDERING | large sudden inflow → immediate consolidation → multi-hop dispersal | single datacenter IP for 24h, then geo-hopping | | FAN_OUT_SMURF | structuring into many sub-threshold amounts | many IPs, one ASN | | ROUND_TRIP | A→B→C→A circular flows | — | | DUSTING | tiny outputs to many addresses | — |

IP realism: sample IPs from actual ranges inside the bundled DB-IP DB so GeoIP resolves. Tag some ASNs as Tor exits, VPN, or bulletproof hosting using an offline snapshot list in data/intel/.

Noise and hardness (so the ML isn't trivial): legit look-alikes (exchange batch payout vs smurfing, Wasabi-style Coinjoin used by privacy-minded legit users), label noise 1-2%, missing network rows for ~20% of TXs, clock jitter ±2 s, shared NAT IPs.

Outputs: data/synth/<name>/{transactions.csv,.json,.xml}, labels.csv (address/tx → typology, cluster_id truth), seeds.csv (a **subset**, ~20%, of illicit wallets given to E4 as "known bad"; the rest stay hidden for evaluation), manifest.json (seed, params, counts, sha256).

CLI: beans synth --n-tx 200000 --illicit-rate 0.04 --seed 42 --formats csv,json,xml --out data/synth/demo

Presets: tiny (5k, for unit tests), demo (200k, for the live demo), bench (1M+, for the performance slide).

6. AI/ML Design (the 4 engines + fusion)

6.0 Feature engineering (shared)

Transaction-level: n_in, n_out, total_in/out (BTC, log), fee, fee-rate, **# equal-value outputs**, output-value entropy, max/min output ratio, round-number flag, has-change-heuristic, script-type mix, address reuse, **time since inputs were created (UTXO age)**, time-to-next-spend of outputs, hour-of-day (UTC and actor-local), is_multisig.

Wallet-level (rolling windows 1h/24h/7d): degree in/out, unique counterparties, recv/sent volume, balance velocity, **holding time**, % outputs spent < 1h, fan-in/fan-out ratio, lifetime, first-seen-to-large-inflow gap, participation in CoinJoin-like TXs, cluster size, **hops to nearest seed**.

Network-level (the correlation): # distinct src_IPs relaying the wallet's TXs, # countries, # ASNs, **ASN type (Tor/VPN/hosting/residential)**, **geo-velocity** (km/h between consecutive broadcasts, i.e. "impossible travel"), non-standard port ratio, first-spy confidence (Δt between first and second relay), off-hours broadcast ratio, IP shared with flagged wallets.

E1 — Entity Clustering (*PS: "common-input-ownership + graph embeddings"*)

1. **CIOH:** all inputs of a TX belong to one entity → union-find. **Exclude CoinJoin-classified TXs** (from E3), or CIOH merges strangers into one super-cluster. Say this out loud in the demo, because it shows you understand the domain.
2. **Change-address heuristic** (conservative): a fresh output with non-round value and the same script type as the inputs → same entity.
3. **Network hint:** addresses whose TXs share a first-spy IP within a time window get a soft link (a weighted edge, never a hard merge).
4. **Embeddings:** node2vec (fast baseline) or **GraphSAGE** (unsupervised link-prediction loss) on the address-transaction graph → 64-dim vectors.
5. **HDBSCAN** on embeddings plus behavioural features → *suggested* merges across CIOH clusters, each with a similarity score. - **Metrics vs ground truth:** Adjusted Rand Index, homogeneity/completeness, cluster purity. Report CIOH-only vs CIOH+embeddings.

E2 — Anomaly Detection (*unsupervised*)

- **Isolation Forest** on tx features and separately on wallet features, with contamination set around the illicit rate.
- Optional: **autoencoder** reconstruction error as a second opinion. Rank-average the two.
- Output: anomaly_score $\in [0,1]$ (percentile-normalised).
- Explain with **SHAP TreeExplainer** on the IForest ("flagged because fee-rate is in the 99.7th percentile and outputs were spent within 3 min").
- **Metrics:** PR-AUC and precision@100 vs labels. The model is unsupervised, but we still evaluate it against the labels.

E3 — Peeling-Chain & Mixing Detection (*supervised, trained on synthetic labels*)

- **Structural extractors** (these produce features, not verdicts):
- *Peel*: TX with 1–2 in / 2 out, one output $\ll$ the other, large output re-spent quickly in the same pattern → walk forward and compute `peel_chain_length`, `peel_ratio_mean`, `hop_interval_mean`.
- *CoinJoin*: `n_equal_outputs`, `denomination_match`, `n_in_distinct_clusters`, input/output count symmetry.
- *Fan-in / fan-out / round-trip*: degree ratios, cycle detection (bounded DFS, length ≤ 5).
- **Model**: LightGBM multi-class {normal, peel, coinjoin, fan_out, fan_in, round_trip} at TX level, with class weights for imbalance.
- **Metrics**: per-class precision/recall/F1 and a confusion matrix. **Report peel-chain recall as a function of chain length.**

E4 — Risk Propagation from Seeds (*PS: "propagate risk scores from seed illicit wallets"*)

- **Baseline (must): Personalised PageRank** on the directed, **value-weighted** wallet flow graph, with teleport restricted to seeds. We also run *reverse* PPR (who funds bad actors).
- **Taint model**: "haircut" proportional taint, where a wallet's taint = the share of its inflow that traces back to seeds, decayed by λ^{hops} . This is interpretable because you can show the path.
- **Learned (should/could): GraphSAGE / GAT** node classifier on the heterogeneous graph (features from §6.0), trained on the labelled synthetic set and validated on the hidden illicit wallets that were *not* seeds. Explain with GNNExplainer.
- **Metrics**: recall@k of hidden illicit wallets given 20% seeds. Plot the curve as the seed fraction goes 5% → 50%.

6.5 Fusion, Scoring & Confidence

- **Meta-model**: logistic regression (or small LightGBM) over [`p_typology` (E3), `anomaly` (E2), `ppr_risk` (E4), `taint`, `gnn_prob`, `cluster_risk` (max/mean over E1 cluster), `network_risk` features].
- **Calibration**: isotonic regression on a validation split, so that "confidence 0.8" really means $\approx 80\%$ of such alerts are truly illicit. Show a reliability diagram in the model card.
- **Risk score** = $100 \times$ calibrated probability.
- **Confidence** = combine model agreement (how many engines concur) with prediction certainty and evidence completeness (e.g. network data present or not).
- **Severity bands**: CRITICAL ≥ 85 · HIGH 65–84 · MEDIUM 40–64 · LOW < 40 .
- **Ranking**: risk $\times$ confidence, deduplicated per cluster: one alert per entity cluster, with member wallets listed.
- **Alert types**: RANSOMWARE_PATTERN, PEEL_CHAIN, MIXING, SEED_PROXIMITY, ANOMALOUS_FLOW, NETWORK_ANOMALY (geo-jump/Tor/VPN), CLUSTER_LINK.

7. Explainability (the R6/R10 requirement)

Every alert JSON carries:

```
{
  "alert_id": "A-000123", "entity": "cluster:C-8812", "type": "PEEL_CHAIN",
  "risk": 91, "confidence": 0.87, "severity": "CRITICAL",
  "reasons": [
    "Part of a 14-hop peeling chain (avg peel 3.1% per hop, avg hop interval 6 min)",
    "2 hops from seed wallet bc1q...9x (known ransomware), taint 62%",
    "Broadcast from 3 countries in 40 min via Tor-exit ASN AS208294 (impossible travel)",
    "Received 4.2 BTC 11 h after wallet creation, held < 2 h"
  ],
  "shap_top": [{"feature": "peel_chain_length", "value": 14, "impact": +0.31}, ...],
  "engine_scores": {"E2_anomaly": 0.93, "E3_peel": 0.96, "E4_ppr": 0.71, "gnn": 0.88},
  "evidence": {"txids": [...], "path_to_seed": [...], "ips": [...], "subgraph_id": "G-123"}
}
```

- **Global:** SHAP summary plot and feature importance in the Model Card page.
- **Local:** SHAP waterfall per alert. A reason template maps (feature, direction) to an English sentence (beans/explain/templates.yaml).
- **Graph evidence:** the shortest value-weighted path to a seed, the peel chain as an ordered path, the CoinJoin TX highlighted. GNNExplainer mask if the GNN is on.
- **Counterfactual line (nice touch):** "Would drop to MEDIUM if not connected to seed X."
- **Model card:** training data, metrics, known limitations (C10H breaks on CoinJoin/PayJoin, synthetic-only training, first-spy is weak against Dandelion++/Tor relaying).

8. Dashboard (screens)

Screen	Content
Overview	KPIs (TXs, wallets, IPs, clusters, alerts by severity), alerts over time, typology donut, top ASNs/countries
Alerts	Ranked table: rank, entity, type, risk, confidence, severity, top reason, #evidence, status. Filters plus search by address/TXID/IP
Entity page	Wallet/cluster/IP profile, engine scores (radar), SHAP waterfall, reasons, linked entities, tx list, investigator notes, and "Mark FP / Confirm / Add to case" buttons
Link graph	Cytoscape. Node shapes: wallet ●, TX ◆, IP ▲, ASN ■. Colour = risk. Seeds get a red ring. Click to expand k-hop. Toggle "peel path" and "path to seed" highlight. Layouts: cose-bilkent / dagre (for chains)
Timeline	Swimlane of fund flow; peel-chain replay slider
Geo map	Offline world GeoJSON; broadcast origins; impossible-travel arcs
Cases	Group alerts, notes, status, Export evidence pack (PDF+JSON, hashed)
Model card	Metrics tables, PR curves, confusion matrix, calibration plot, SHAP summary
Ingest	Upload a CSV/JSON/XML or pick a folder, set a column mapping, watch the progress bar, see the quarantine report, re-score

UI rules: dark theme (investigator style), monospace addresses with copy buttons, every number clickable down to its evidence.

9. API (FastAPI)

```
GET /api/health
POST /api/ingest (multipart file | path) → job_id
GET /api/jobs/{id}
POST /api/pipeline/run (enrich → graph → engines → score)
GET /api/stats/overview
GET /api/alerts?severity=&type=&status=&q=&limit=&offset=
GET /api/alerts/{id} (full explanation + evidence)
PATCH /api/alerts/{id} (status, notes, feedback label)
GET /api/entity/wallet/{addr} | /ip/{ip} | /cluster/{id} | /tx/{txid}
GET /api/graph/subgraph?center=&hops=&min_risk=
GET /api/graph/path?from=&to=seed
POST /api/seeds (upload seed list) → re-propagate
GET /api/model/card
POST /api/cases · GET /api/cases/{id} · GET /api/cases/{id}/export?fmt=pdf|json
GET /api/audit
```

Bind to 127.0.0.1 by default. Every mutating call writes to `audit_log`.

10. Repository Layout

```
BEANS/
├── MASTER_ROADMAP.md          ← this file
├── README.md                  quickstart (offline), screenshots
├── Makefile                    setup-offline · synth · pipeline · serve · demo · test · bundle
├── pyproject.toml
├── beans/
│   ├── cli.py                 typer entrypoint: beans <cmd>
│   ├── config.py              pydantic-settings, .env
│   ├── schema.py              canonical record models
│   ├── ingest/                csv.py · json.py · xml.py · mapping.py · validate.py
│   ├── enrich/                geoip.py · asn_type.py · address.py
│   ├── store/                 duck.py · ddl.sql
│   ├── graph/                 build.py · firstspy.py · export_neo4j.py
│   ├── features/              tx.py · wallet.py · network.py
│   ├── engines/               e1_cluster.py · e2_anomaly.py · e3_peelmix.py · e4_propagate.py · gnn.py
│   ├── score/                 fuse.py · calibrate.py · alerts.py
│   ├── explain/               shap_explain.py · reasons.py · templates.yaml · evidence.py
│   ├── report/                pdf.py · templates/case.html.j2
│   ├── synth/                 actors.py · typologies.py · ledger.py · ipgen.py · writer.py
│   └── api/                   main.py · routes/*.py
├── ui/                         React+Vite+TS (src/), built into ui/dist (committed for offline)
├── models/                     trained artifacts (*.pkl, *.pt) + model_card.json
├── data/
│   ├── geoip/                 dbip-country-lite.mmdb · dbip-asn-lite.mmdb (+ LICENSE)
│   ├── intel/                 tor_exits.csv · hosting_asns.csv · vpn_asns.csv (offline snapshots, dated)
│   ├── seeds/                 illicit_seeds.csv
│   ├── synth/                 generated datasets (gitignored except tiny/)
│   └── samples/               tiny CSV/JSON/XML examples
├── notebooks/                 EDA, model experiments
├── tests/                      unit + e2e (tiny dataset)
├── docs/
│   ├── TECHNICAL_WRITEUP.md   R10 deliverable
│   ├── DATA_FORMATS.md
│   ├── ARCHITECTURE.md
│   └── archive/
├── scripts/                    approach (1).md, setup.md, SIH_PS_26146.pdf
└── .github/workflows/ci.yml     build_offline_bundle.sh · fetch_geoip.sh (run ONCE online)
```

10.1 Offline strategy (R8)

1. **Online once:** `scripts/fetch_geoip.sh` pulls DB-IP Lite, and `pip download -r requirements.txt -d wheelhouse/` pulls wheels. Build the UI with `npm ci && npm run build`.
2. **Bundle:** `make bundle` → `beans-offline-<ver>.tar.gz` = code + wheelhouse + `ui/dist` + `mmdb` + models + demo dataset.
3. **Target machine:** `tar xzf ... && make setup-offline` (`pip install --no-index --find-links wheelhouse`) → `make demo`.
4. Optional: `docker build` → `docker save beans:1.0 > beans.tar` → `docker load < beans.tar` offline.

5. **Test it for real:** a fresh Ubuntu VM with the network adapter disabled. Do this at least twice before the finale.
 6. Pin CPU-only torch wheels (smaller, no CUDA surprises).
-

11. Build Plan (timeline)

Team of 6 roles: **A** Data/Synth · **B** Ingest/Store/Enrich · **C** Graph + E1/E4 · **D** E2/E3 + Fusion/Explain · **E** Frontend · **F** API/DevOps/Docs/PPT (PM).

Phase 0 — Idea submission / PPT (Week 0, ~3 days)

- ☐ Finalise this roadmap; architecture diagram as an image; the 4-engine story
- ☐ SIH idea PPT: problem → solution → architecture → ML engines → explainability → offline → impact → feasibility → team
- ☐ Tiny proof: generate 5k synthetic rows, show one peel chain found in a notebook. A screenshot in the PPT carries a lot of weight.

Phase 1 — Foundations (Week 1)

- ☐ A: `beans synth v1`: ledger, retail, exchange, 3 typologies (peel, coinjoin, ransomware), CSV/JSON/XML writers, labels
- ☐ B: canonical schema, 3 streaming parsers, validation + quarantine, DuckDB DDL, GeoIP enrichment
- ☐ F: repo skeleton, Makefile, CI, ruff/pytest, FastAPI health, README
- ☐ E: Vite+React scaffold, layout, mock alert table from a static JSON
- **Exit:** `beans synth tiny` & `beans ingest` works end to end for all 3 formats

Phase 2 — Graph + engines v1 (Week 2)

- ☐ C: graph build, first-spy attribution, CIOH union-find, PPR from seeds
- ☐ D: feature library (tx/wallet/network), Isolation Forest, peel/coinjoin extractors, LightGBM v1
- ☐ A: remaining typologies + noise/hardness knobs; `demo` preset
- ☐ E: alert table and entity page wired to the API; Cytoscape graph v1
- **Exit:** `make pipeline` produces a ranked alert list from the synthetic data

Phase 3 — Fusion, explainability, UI (Week 3)

- ☐ D: fusion meta-model, isotonic calibration, SHAP, reason templates, alert JSON
- ☐ C: node2vec/GraphSAGE embeddings + HDBSCAN; taint paths; path-to-seed API
- ☐ E: graph expand/filter/highlight, timeline, geo map, SHAP waterfall
- ☐ F: evaluation harness `beans eval` → metrics JSON → model card page
- **Exit:** a full demo flow works in the UI

Phase 4 — Hardening & "wow" (Week 4)

- ☐ Cases, evidence PDF export, audit log, feedback loop (S3-S5)
- ☐ 1M-row benchmark and profiling (polars/DuckDB vectorisation)
- ☐ GNN (C1) if E4 baseline is stable
- ☐ Offline bundle + clean-VM test #1
- ☐ Custom-column-mapping test with an unseen file layout (NTRO may give their own data)

Phase 5 — Polish (Week 5 → finale)

- ☐ Technical write-up (§12), README with GIFs, demo video backup (local MP4)
- ☐ Clean-VM offline test #2; freeze dependencies; tag v1.0
- ☐ Rehearse the demo script 5+ times; prepare Q&A (§14)

Grand Finale (36 h) plan

Judges often hand you a new dataset or ask for a change. Reserve time for that:

- **0-4 h:** ingest their data via `--mapping`, run the pipeline, fix parsing edge cases
- **4-16 h:** implement mentor feedback (typically a new typology, a new view, or a threshold UI)
- **16-28 h:** retrain on combined data, regenerate the model card, polish UI
- **28-34 h:** freeze, rehearse, backup video
- **Mentor rounds:** always show *live* numbers from *their* data

12. Technical Write-up Outline (`docs/TECHNICAL_WRITEUP.md`, 6-8 pages)

1. Problem & threat model (ransomware, darknet, laundering; why network+chain correlation helps)
2. Data: schema, 3 formats, GeolIP source (DB-IP Lite, CC-BY 4.0, attribution), **synthetic generator design & typologies**
3. Architecture diagram + offline design
4. Graph model and first-spy correlation
5. Models: E1-E4 + fusion. **Why these models** (IForest: unsupervised with no labels in real life; LightGBM: tabular SOTA and SHAP-friendly; PPR: fits seed propagation directly and can be explained; GNN: learns the graph context)
6. **Explainability method:** SHAP (local and global), reason templates, path evidence, GNNExplainer, calibration
7. Results: per-engine metrics, ablation (**with vs without network layer**, CIOH vs CIOH+embeddings), seed-fraction curve, performance benchmark
8. Limitations & ethics: synthetic-only, CIOH breaks on CoinJoin/PayJoin, first-spy weakened by Dandelion++/Tor, a human always reviews before action, data-protection handling
9. How to run (offline)

13. Evaluation & Metrics (what goes on the "Results" slide)

What	Metric	Target (synthetic <code>dataset</code>)
Ingest	rows/sec, 1M-row wall time	$\geq 50k$ rows/s; $1M < 60$ s
E1 clustering	ARI, purity	$ARI \geq 0.8$ (CIOH+emb > CIOH-only)
E2 anomaly	PR-AUC, P@100	$P@100 \geq 0.7$
E3 peel/mix	macro-F1, peel recall (len ≥ 5)	$F1 \geq 0.85$; recall ≥ 0.9
E4 propagation	recall@200 of hidden illicit, 20% seeds	≥ 0.75
Fusion	PR-AUC, precision@50, calibration ECE	$P@50 \geq 0.9$; $ECE < 0.05$
Ablation	Δ PR-AUC with network features	show positive gain
End-to-end	full pipeline on 200k rows	< 3 min on 8-core/16 GB

Targets are goals, not claims. Report the real numbers honestly with seeds fixed (--seed 42) so they can be reproduced.

14. Demo Script (7 minutes, Wi-Fi OFF on screen)

1. **(30s)** Show the airplane-mode icon. `make demo` starts.
2. **(60s)** Ingest: drop CSV, JSON, and XML versions of the dataset; show identical counts and the quarantine report. GeoIP coverage 99%.
3. **(60s)** Overview: 200k TXs, N clusters, alerts by severity.
4. **(90s)** Top alert: *CRITICAL · Peel chain linked to ransomware seed*. Read the plain-English reasons. Show the SHAP waterfall.
5. **(90s)** Link graph: expand; highlight the peel path hop by hop on the timeline; show the IP ▲ in a Tor-exit ASN and the impossible-travel arc on the map. **This is the network ↔ chain correlation moment.**
6. **(45s)** Upload 3 new seed wallets → risk re-propagates live → new alerts appear.
7. **(30s)** Mark one alert as a false positive (feedback loop). Export the case → PDF evidence pack with the input file's SHA-256.
8. **(30s)** Model card: metrics, the network-layer ablation gain, calibration plot. Close.

15. Judge Q&A prep

- "How is this not just rules?" → rules produce features; LightGBM/IForest/GNN/meta-model make the decisions; show the feature importances.
- "How do you get labels in real life?" → seeds from LE cases + unsupervised E2 + analyst feedback loop (active learning). The synthetic data is only for bootstrap and evaluation.
- "CIOH breaks with CoinJoin." → we detect CoinJoin first (E3) and exclude it from CIOH.
- "Can network data really identify the source?" → first-spy is probabilistic (Biryukov et al. 2014; weaker since Dandelion++ / Tor), so we use it as a *weighted* feature with a confidence value and never as proof.
- "Scale?" → the benchmark slide, DuckDB/Parquet columnar storage, and the Neo4j export path.
- "Why is it offline?" → the NTRO requirement, air-gapped use, and no data leaves the machine. There's an audit log too.
- "Why DB-IP and not MaxMind?" → no license key needed and redistributable (CC-BY). The reader also accepts GeoLite2 `.mmdb` files.

16. Risks & Mitigations

Risk	Mitigation
Synthetic data too easy → unrealistic 99% scores	Hardness knobs, legit look-alikes, label noise; report numbers on the <i>hard</i> preset
Judges bring an unfamiliar file format	--mapping YAML, tolerant parsers, test with 3 different layouts
PyG / torch install fails offline	CPU wheels in the wheelhouse; the GNN is a COULD, and PPR baseline always works
Demo laptop slow	Precomputed demo DB committed as a release asset; the pipeline can also run live
Graph view is an unreadable hairball	Always subgraph-centred ($k\text{-hop} \leq 2$, top-N by risk), dagre layout for chains

Risk	Mitigation
Scope creep	MoSCoW above. Finish all MUST items before any SHOULD item.

17. Immediate Next Actions (this week)

1. ☐ Push this file + move old docs into `docs/archive/` → first commit to `origin/main`
2. ☐ Create the repo skeleton (\$10) + Makefile + CI
3. ☐ `scripts/fetch_geoip.sh` → download DB-IP Lite country + ASN mmdb (the one online step)
4. ☐ `beans synth --preset tiny` with peel + coinjoin + ransomware and all 3 writers
5. ☐ Parsers for all 3 formats + DuckDB load + first notebook showing a detected peel chain
6. ☐ Assign owners A-F; set up a GitHub Project board with the M/S/C items as issues